МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Заитова Роза Рустамовна
(подпись) (фамилия, имя, отчество)

Группы __Ипо-41__

Руководитель практической подготовки от профильной организации

_____Декан математического факультета ПГГПУ_____
(должность)

_____ _____Скорнякова Анна Юрьевна_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____Берестов Дмитрий Борисович__
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ....3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ....5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии....5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ....8
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении....8
Тема 2.2. Ознакомление с должностными и функциональными обязанностями....10
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ....12
Тема 3.1. Ознакомление: с организацией информационного обеспечения подразделения....12
Тема 3.2. Выполнение производственных заданий....16
3.2.1. использование выбранной системы контроля версий....16
3.2.2. разработка программного обеспечения....17
3.2.3. Использование методов для получения кода с заданной функциональностью и степенью качества....31
3.2.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования....33
3.2.5. Выполнение отладки программного модуля с использованием специализированных программных средств....34
3.2.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения....35
3.2.7. Интегрирование программных модулей....40
3.2.8. Верификация и аттестация программного обеспечения....42
ЗАКЛЮЧЕНИЕ....45
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ....47

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Практика проходила на базе математического факультета федерального государственного бюджетного образовательного учреждения высшего образования «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ). Факультет осуществляет подготовку специалистов в области математики, информатики и ИТ-технологий, что делает его идеальной площадкой для прохождения практики по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения; интеграция модулей в программное обеспечение; отладка программного модуля с использованием специализированных программных средств; разработка тестовых наборов и

тестовых сценариев; инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
7. Планировать и реализовывать собственное профессиональное и личностное развитие;
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
10. Использовать информационные технологии в профессиональной деятельности
11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Первый день практики начался с организационных мероприятий на математическом факультете ПГГПУ.

Я была ознакомлена с общими функциональными обязанностями лаборанта кафедры информатики и вычислительной техники, к которым относятся: обеспечение учебного процесса (подготовка оборудования), техническое обслуживание компьютерной техники, помощь преподавателям в проведении занятий, документооборот.

Был проведен подробный инструктаж по технике безопасности. Основные правила включают:

- Неукоснительное соблюдение правил пожарной безопасности, знание расположения огнетушителей и плана эвакуации.
- Запрет на самостоятельное вскрытие и ремонт компьютерной техники, находящейся под напряжением.
- Обязательное отключение оборудования от электросети при его перемещении или подключении новых устройств.
- Недопущение нахождения жидкостей, пищи на рабочем месте вблизи системных блоков.
- При возникновении нештатной ситуации (запах гари, искрение) немедленно обесточить оборудование и сообщить руководителю.

Мое рабочее место было организовано в компьютерном классе, оснащенном современной техникой, что полностью соответствовало требованиям для выполнения поставленной задачи по разработке программного обеспечения (см. Рисунок 1).

Рисунок 1 — Рабочее место в компьютерном классе математического факультета ПГГПУ.

Конфигурация оборудования и программного обеспечения: Для выполнения работ использовался персональный компьютер со следующими характеристиками и набором ПО:

- Аппаратное обеспечение: Процессор Intel Core i5, оперативная память 16 ГБ, твердотельный накопитель (SSD) 512 ГБ.
- Операционная система: Microsoft Windows 10 Pro (лицензионная корпоративная версия).
- Прикладное программное обеспечение:
    1. Офисный пакет: Microsoft Office Standard 2019 (Word, Excel — для оформления отчета).
    2. Среда разработки: Microsoft Visual Studio 2022 Community Edition (основной инструмент для написания кода на C# и разработки WPF-интерфейса).
    3. Система управления базами данных (СУБД): PostgreSQL, а также инструмент администрирования pgAdmin 4.
    4. Система контроля версий: Git, интегрированная с Visual Studio.

5. Вспомогательное ПО: Браузер Google Chrome (для поиска информации), 7-Zip (архиватор), Adobe Acrobat Reader (для чтения документации).

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Математический факультет ПГГПУ является структурным подразделением университета, осуществляющим подготовку бакалавров и магистров по направлениям «Математика», «Информатика», «Педагогическое образование». История факультета насчитывает более 50 лет, за это время было подготовлены высококвалифицированных специалистов, работающих в школах, вузах, научных и коммерческих организациях не только Пермского края, но и других регионов России.

Факультет имеет развитую инфраструктуру, включающую:

- **Деканат** – административный центр, где решаются вопросы организации учебного процесса, распределения нагрузки, работы с контингентом студентов.
- **Кафедры** – основные учебно-научные подразделения. На факультете функционируют две кафедры: кафедра высшей математики и методики обучения математике, а также кафедра информатики и вычислительной техники (именно к этой кафедре я был прикреплен для прохождения практики).
- **Компьютерные классы** – специализированные аудитории, оснащенные современной вычислительной техникой для проведения занятий по информатике, программированию, компьютерному моделированию.
- **Учебно-методический кабинет** – место хранения учебно-методической литературы, курсовых и дипломных работ, архива кафедр.
- **Научно-методическая лаборатория** – площадка для проведения исследований в области методики преподавания математики и информатики.
- **Кабинет истории математики и математического образования** – уникальное пространство, где собраны материалы по истории развития

математической мысли, биографии выдающихся ученых, старинные учебники.

Режим работы факультета: с 9:00 до 18:00 (пн-пт), перерыв на обед с 13:00 до 14:00. Рабочее место практиканта было организовано в компьютерном классе (ауд. 305), оснащенном современной вычислительной техникой. В аудитории установлено 15 рабочих станций для студентов и одна рабочая станция преподавателя, объединенных в локальную сеть с доступом в Интернет. Все компьютеры имеют одинаковую конфигурацию, что обеспечивает единообразие программной среды и упрощает техническое обслуживание.

Таблица 1 – Характеристики оборудования компьютерного класса.

Компонент	Характеристика	Назначение
Процессор	Intel Core i5-10400, 2.9 ГГц (6 ядер, 12 потоков)	Обеспечение высокой производительности при компиляции кода, работе с виртуальными машинами, многозадачности
Оперативная память	16 ГБ DDR4 (2 планки по 8 ГБ, двухканальный режим)	Достаточный объем для работы с современными IDE, СУБД, несколькими открытыми проектами
Накопитель	SSD 512 ГБ (NVMe)	Высокая скорость загрузки ОС и приложений, быстрая компиляция, моментальный запуск
Монитор	23.8" Full HD (1920x1080), IPS-матрица	Комфортная работа с кодом, документацией, визуальными элементами интерфейса
Видеокарта	Интегрированная Intel UHD Graphics 630	Достаточно для 2D-графики, разработки WPF-приложений, работы с офисными приложениями
Сетевое оборудование	Коммутатор D-Link DGS-1100-24, скорость 1 Гбит/с	Обеспечение высокоскоростного обмена данными между компьютерами и доступа в Интернет

Компонент	Характеристика	Назначение
МФУ	Canon i-SENSYS MF443dw	Печать, сканирование, копирование документов (скорость печати до 38 стр/мин)

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Мною была изучена должностная инструкция лаборанта кафедры информатики и ВТ, утвержденная деканом математического факультета. Документ содержит следующие разделы:

- Общие положения
- Функциональные обязанности
- Права
- Ответственность
- Взаимоотношения (связи по должности)

Основные обязанности лаборанта включают:

1. **Обеспечение работоспособности компьютерной техники и ПО:**

   - Проверка работоспособности оборудования перед началом занятий.
   - Установка и настройка программного обеспечения по заявкам преподавателей.
   - Обновление операционных систем и прикладных программ.
   - Мелкий ремонт и техническое обслуживание (чистка от пыли, замена периферии).
   - Взаимодействие с техническими службами университета при серьезных поломках.

2. **Помощь преподавателям в подготовке материалов к занятиям:**

   - Распечатка раздаточных материалов, методических пособий.
   - Подготовка презентаций, видеоматериалов.

- Копирование и сканирование документов.
- Подбор литературы в ЭБС.

3. **Ведение документации кафедры:**

- Оформление протоколов заседаний кафедры.
- Подготовка отчетов по учебной и научной работе.
- Ведение учета учебной нагрузки преподавателей.
- Регистрация и хранение курсовых и дипломных работ.

4. **Работа с электронной информационно-образовательной средой (ЭИОС) университета:**

- Размещение учебно-методических материалов.
- Помощь преподавателям в заполнении электронных журналов.
- Консультирование студентов по работе с ЭИОС.

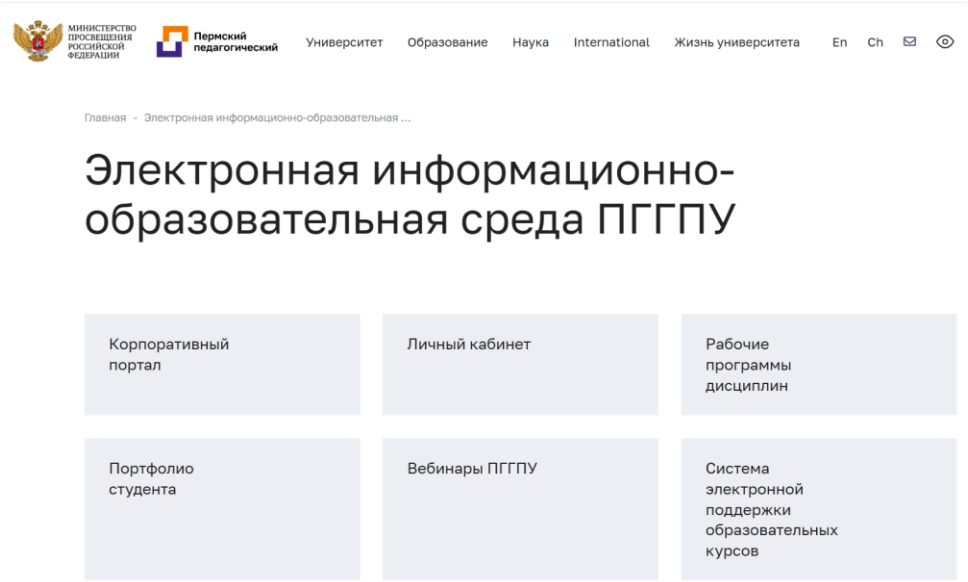

Рисунок 2 – Интерфейс ЭИОС ПГГПУ.

Взаимодействие с другими подразделениями:

- С деканатом – по вопросам документации, отчетности.
- С учебно-методическим управлением – по вопросам учебных планов, программ.
- С библиотекой – по вопросам обеспечения учебной литературой.
- С административно-хозяйственной частью – по вопросам ремонта и обслуживания.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление: с организацией информационного обеспечения подразделения

В ходе прохождения производственной практики на математическом факультете ПГГПУ мною было проведено детальное изучение организации информационного обеспечения деятельности подразделения. Информационное обеспечение представляет собой совокупность процессов сбора, хранения, обработки и передачи информации, необходимой для эффективного функционирования факультета и обеспечения учебного процесса. Качество информационного обеспечения напрямую влияет на качество образования и скорость принятия управленческих решений.

Математический факультет обладает развитой информационной инфраструктурой, включающей аппаратное обеспечение (компьютерное и сетевое оборудование), программное обеспечение (системное и прикладное ПО), информационные ресурсы (базы данных, электронные библиотеки), а также организационные документы, регламентирующие работу с информацией.

Аппаратное обеспечение

На факультете функционирует три компьютерных класса, расположенных в аудиториях 305, 307 и 309. Аудитория 305, где было организовано мое рабочее место, представляет собой типовой компьютерный класс, оснащенный 15 рабочими станциями для студентов и одной рабочей станцией для преподавателя. Все компьютеры объединены в локальную сеть и имеют доступ в Интернет.

Аудитория 309 является серверной, где размещено сетевое и серверное оборудование: сервер HP ProLiant DL380 Gen10 с 64 ГБ RAM и дисками в RAID10, два коммутатора D-Link, источник бесперебойного питания APC.

Программное обеспечение

На компьютерах факультета установлено следующее лицензионное программное обеспечение:

Таблица 2 – Программное обеспечение компьютерных классов.

Категория	Наименование ПО	Назначение
Операционные системы	Microsoft Windows 10 Pro	Базовая ОС
	Ubuntu 20.04 LTS	Изучение Linux
Офисное ПО	Microsoft Office Standard 2019	Работа с документами
Средства разработки	Microsoft Visual Studio 2022	Разработка на C#
	Visual Studio Code	Редактирование кода
	PyCharm	Разработка на Python
СУБД	PostgreSQL 16	Сервер БД
	pgAdmin 4	Администрирование PostgreSQL
Математическое ПО	MathCAD, MATLAB	Математические расчеты
Веб-браузеры	Google Chrome, Firefox	Доступ в Интернет
Антивирус	Kaspersky Endpoint Security	Защита от вредоносного ПО

Информационные системы университета

В своей деятельности факультет использует следующие информационные системы:

Электронная информационно-образовательная среда (ЭИОС) ПГГПУ – портал, объединяющий все образовательные ресурсы университета. Через ЭИОС студенты получают доступ к:

- рабочим программам дисциплин;
- электронным учебным пособиям;
- заданиям для самостоятельной работы;
- тестовым материалам;
- электронному расписанию;
- своим оценкам и рейтингу.

Автоматизированная система «Нагрузка» предназначена для планирования и учета учебной нагрузки профессорско-преподавательского состава. Система позволяет:

- распределять учебные поручения;
- формировать индивидуальные планы преподавателей;
- учитывать выполненную нагрузку;
- формировать отчеты по кафедре и факультету.

Электронно-библиотечные системы (ЭБС):

- ЭБС «IPRbooks» (www.iprbookshop.ru);
- ЭБС «Znanium.com» (znanium.com);
- ЭБС «Лань» (e.lanbook.com).

Организация защиты информации

В университете действует политика информационной безопасности, регламентирующая правила работы с информационными системами и данными.

Основные меры защиты:

1. **Аутентификация и управление доступом:**

- Каждый пользователь имеет уникальную учетную запись.
- Доступ к ресурсам предоставляется в соответствии с должностными обязанностями.

- Пароли должны соответствовать требованиям сложности (не менее 8 символов, буквы разного регистра, цифры).

2. **Защита от вредоносного ПО:**

- На всех компьютерах установлен Kaspersky Endpoint Security.
- Обновление антивирусных баз происходит автоматически.
- Запрещено использование нелицензионного ПО.

3. **Защита персональных данных:**

- Обработка персональных данных осуществляется в соответствии с ФЗ-152.
- Доступ к базам данных с персональными данными ограничен.
- Работники подписывают обязательства о неразглашении.

4. **Резервное копирование:**

- Ежедневное резервное копирование критически важных данных.
- Хранение резервных копий на отдельных носителях.
- Регулярное тестовое восстановление.

Техническое обслуживание

В задачи лаборанта кафедры входит техническое обслуживание оборудования и поддержка пользователей. Основные виды работ:

Ежедневное обслуживание:

- Включение и проверка работоспособности оборудования;
- Контроль наличия расходных материалов;
- Выключение оборудования после занятий.

Текущее обслуживание:

- Установка и настройка ПО по заявкам преподавателей;
- Обновление операционных систем и прикладных программ;
- Диагностика и устранение мелких неполадок;
- Консультирование преподавателей и студентов;
- Взаимодействие с отделом информатизации при серьезных поломках.

Тема 3.2. Выполнение производственных заданий

3.2.1. использование выбранной системы контроля версий

В процессе разработки программного модуля использовалась система контроля версий Git. Git — это распределенная система управления версиями, которая позволяет отслеживать изменения в исходном коде, возвращаться к предыдущим версиям, работать в команде и хранить историю разработки.

Основные команды Git, использованные в работе:

Таблица 3 – Основные команды Git.

Команда	Назначение
git init	Инициализация локального репозитория
git add .	Добавление всех измененных файлов в индекс
git commit -m "сообщение"	Фиксация изменений с комментарием
git remote add origin [URL]	Привязка локального репозитория к удаленному
git push -u origin main	Отправка изменений на удаленный репозиторий
git pull	Получение актуальных изменений из удаленного репозитория
git status	Проверка состояния файлов

Проект был размещен в удаленном репозитории на GitHub (см. Рисунок 3), что обеспечивает доступность кода, резервное копирование и возможность совместной разработки. Репозиторий назван с учетом фамилии студента: Zaitova_MasterPol.

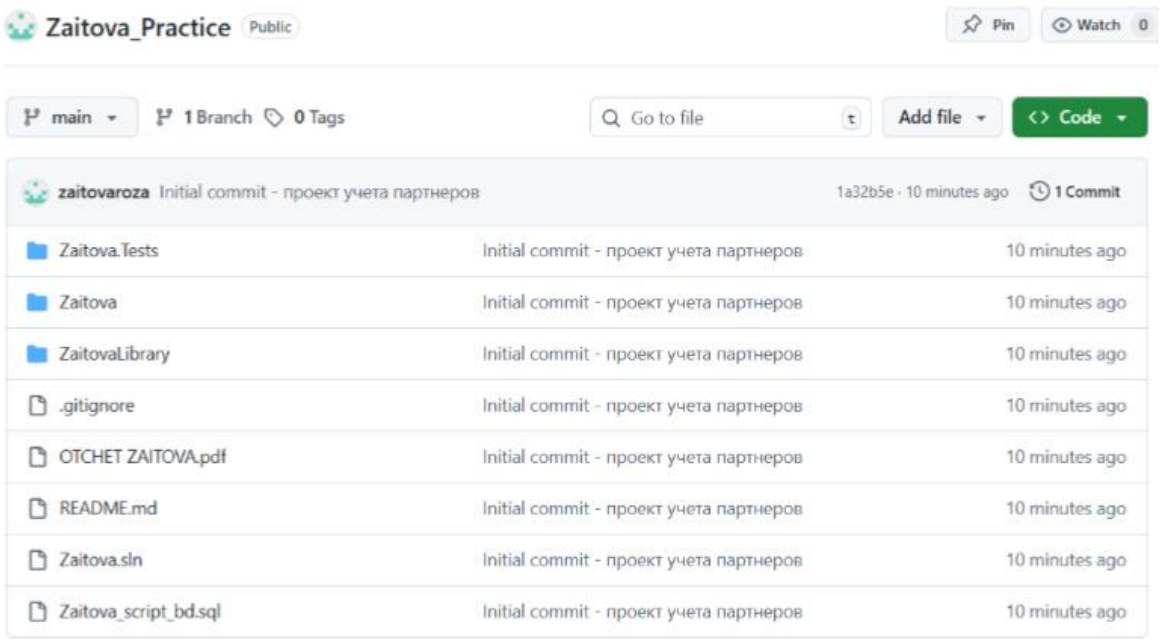

Рисунок 3 – Репозиторий проекта на GitHub.

Использование Git позволило:

- Фиксировать этапы разработки с комментариями.
- При необходимости откатываться к предыдущим версиям.
- Контролировать версионность файлов.
- Обеспечить сохранность кода при сбоях оборудования.

3.2.2. разработка программного обеспечения

Выбор модели разработки

Для создания программного модуля была выбрана итерационная модель разработки. Данный выбор обоснован следующими факторами:

1. Постепенное наращивание функциональности – возможность реализовывать и тестировать функционал поэтапно (сначала просмотр списка, затем добавление, потом редактирование и т.д.).
2. Гибкость – возможность вносить изменения в требования в процессе разработки (например, изменение дизайна истории заказов).
3. Минимизация рисков – на каждой итерации проводится тестирование, что позволяет выявлять ошибки на ранних стадиях.

4. Вовлеченность заказчика – на каждой итерации можно демонстрировать промежуточные результаты руководителю практики и корректировать дальнейшую работу.

Инструменты разработки

Для создания программного продукта использовались следующие инструменты:

Таблица 4 – Инструменты разработки.

Инструмент	Назначение	Обоснование выбора
Microsoft Visual Studio 2022 Community	Среда разработки	Бесплатная, мощная, поддерживает все современные технологии .NET, встроенные средства отладки и рефакторинга
C#	Язык программирования	Объектно-ориентированный, типобезопасный, современный язык с широкими возможностями
WPF (Windows Presentation Foundation)	Технология создания интерфейса	Позволяет создавать гибкие, стилизованные интерфейсы с использованием XAML и привязки данных (Data Binding)
PostgreSQL 16	Система управления базами данных	Бесплатная, надежная, соответствует требованиям ТЗ, поддерживает сложные запросы и транзакции
Entity Framework Core 8	ORM (Object-Relational Mapping)	Упрощает работу с БД, позволяет работать с данными как с объектами C#, автоматически генерирует SQL-запросы

Инструмент	Назначение	Обоснование выбора
Git	Система контроля версий	Позволяет отслеживать изменения, работать с удаленными репозиториями
GitHub	Хостинг репозиториев	Бесплатный, широко распространен, удобен для демонстрации кода

Среда программирования

Разработка велась в среде Microsoft Visual Studio 2022 Community. Данная среда предоставляет широкие возможности для разработки на платформе .NET:

- Редактор кода с подсветкой синтаксиса, IntelliSense (автодополнение), рефакторингом.
- Визуальный конструктор XAML для создания интерфейса WPF (см. Рисунок 4).
- Встроенный отладчик – позволяет устанавливать точки останова, просматривать значения переменных, выполнять пошаговую отладку.
- Интеграция с Git – возможность выполнять коммиты, push/pull прямо из IDE.
- Менеджер пакетов NuGet – для установки сторонних библиотек (EF Core, Npgsql и др.).

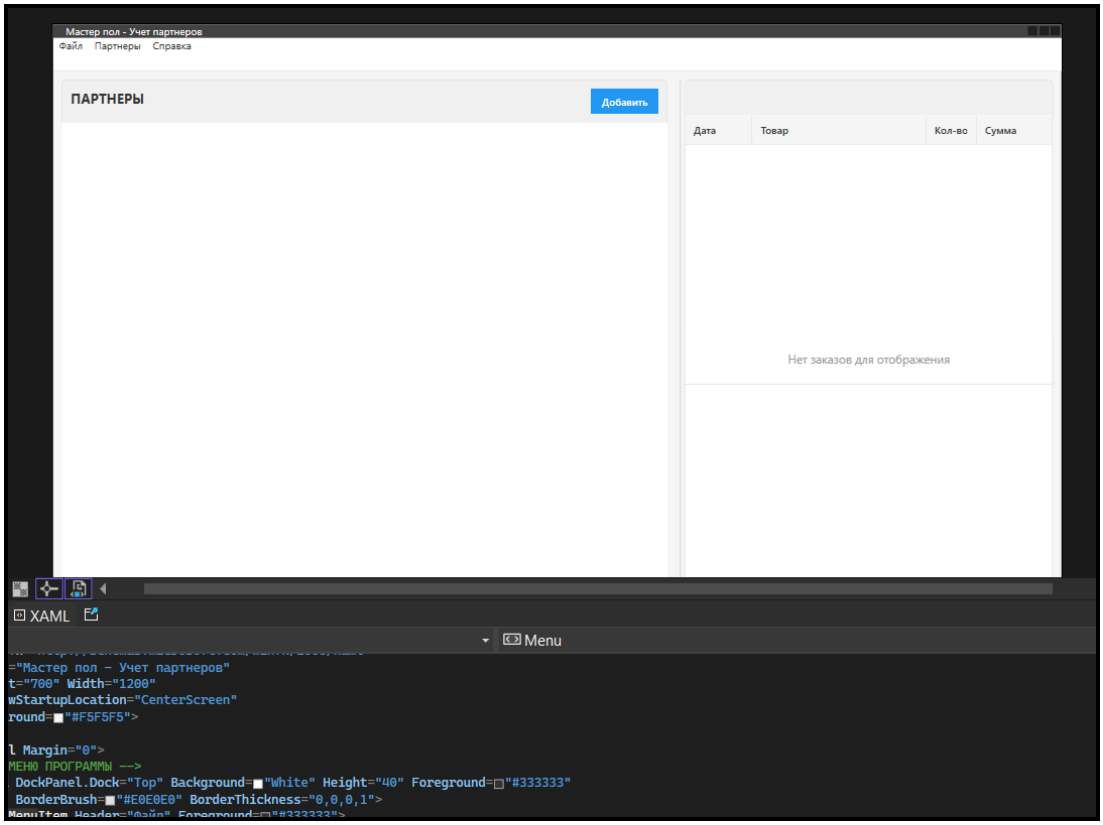

Рисунок 4 – Разработка интерфейса в Visual Studio 2022.

Разработка программного модуля согласно требованиям

В соответствии с заданием требовалось разработать подсистему для работы с партнерами компании «Мастер пол». Было реализовано три основных функции:

1. Просмотр списка партнеров (главное окно, левая колонка).
2. Добавление/редактирование данных о партнере (отдельное окно PartnerEditWindow).
3. Просмотр истории реализации продукции партнером (правая колонка главного окна).

Проект назван с учетом фамилии студента: Zaitova_MasterPol, что соответствует требованиям методических указаний.

Этапы разработки ПО:

1. Выбор шаблона проекта

Был создан новый проект типа WPF Application с именем Zaitova_MasterPol. Также создана библиотека классов ZaitovaLibrary для вынесения логики работы с данными (репозитории, сервисы, модели). Структура решения представлена на Рисунке 5.

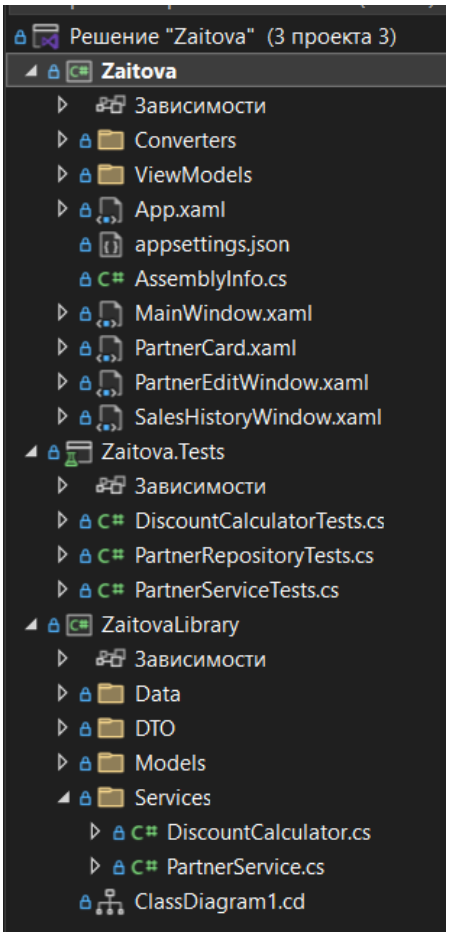

Рисунок 5 – Структура решения в Visual Studio.

2. Составление схемы информационной базы

На основе анализа предметной области была спроектирована база данных, состоящая из следующих таблиц:

Таблица 5 – Структура базы данных.

Таблица	Назначение	Основные поля
partner_types_zaitova	Справочник типов партнеров	id, name, description

Таблица	Назначение	Основные поля
partners_zaitova	Данные о партнерах	id, type_id, company_name, legal_address, inn, director_fullname, phone, email, rating, created_at, updated_at
products_zaitova	Справочник продукции	id, article, name, description, min_price, unit
sales_history_zaitova	История продаж	id, partner_id, product_id, quantity, sale_date

Схема базы данных соответствует 3 нормальной форме (3НФ), обеспечена ссылочная целостность с помощью внешних ключей. ER-диаграмма базы данных представлена на Рисунке 6.

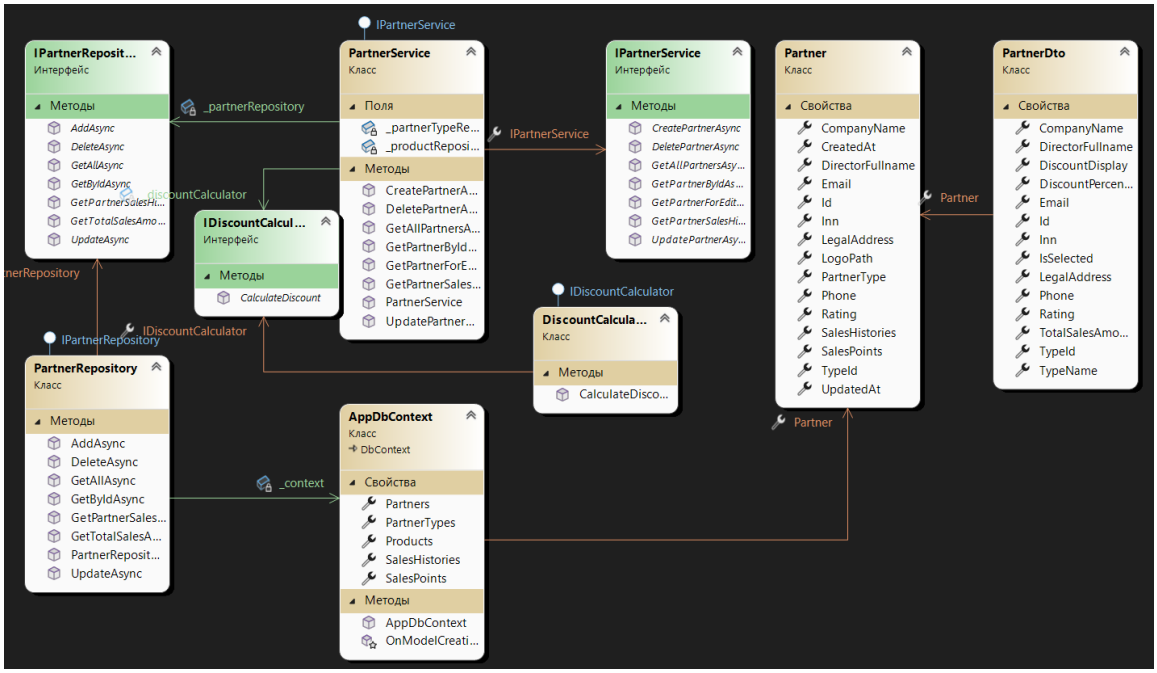

Рисунок 6 – ER-диаграмма базы данных.

Скрипт создания таблицы partners_zaitova:

```
CREATE TABLE app.partners_zaitova (
    id SERIAL PRIMARY KEY,
    type_id INTEGER NOT NULL REFERENCES app.partner_types_zaitova(id) ON
DELETE RESTRICT,
    company_name VARCHAR(255) NOT NULL,
    legal_address TEXT,
    inn VARCHAR(12) UNIQUE,
```

```
        director_fullname VARCHAR(255),
        phone VARCHAR(20),
        email VARCHAR(100),
        rating INTEGER NOT NULL DEFAULT 0 CHECK (rating >= 0 AND rating <=
100),
        logo_path TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
```

На основе анализа предметной области была спроектирована база данных в СУБД PostgreSQL. База данных состоит из четырех основных таблиц, связанных отношением "один-ко-многим": справочник типов партнеров (partner_types_zaitova), таблица партнеров (partners_zaitova), справочник продукции (products_zaitova) и таблица истории продаж (sales_history_zaitova).

После создания таблиц они были заполнены тестовыми данными для проверки работоспособности приложения. На Рисунке 7 представлено содержимое таблицы партнеров после заполнения.

id [PK] integer	type_id integer	company_name character varying (255)	legal_address text	inn character varying (12)	director_fullname character varying (255)	phone character varying (20)	email character varying (100)	rating integer
1	1	ООО "ПермьПаркет"	г. Пермь, ул. Мира, 25	5904123456	Кузнецов Андрей Петрович	+7(342)111-22-33	info@permparket.ru	94
2	2	ИП Соболева Е.В.	г. Пермь, ул. Ленина, 52	5904234567	Соболева Елена Владимировна	+7(912)333-44-55	soboleva@mail.ru	72
3	1	ООО "УралПол"	г. Пермь, Комсомольский пр....	5904345678	Мальцев Сергей Александрович	+7(342)555-66-77	uralpol@bk.ru	85
4	3	ЗАО "СтройКомплект"	г. Березники, ул. Пятилетки, 18	5904456784	Воронова Татьяна Игоревна	+7(342)777-88-99	stroykom@mail.ru	91
6	1	ООО "ПармаСтрой"	г. Кунгур, ул. Карла Маркса, 12	5904678901	Одинцов Павел Валерьевич	+7(342)999-00-11	parma@yandex.ru	79
7	2	ИП Козлова Н.С.	г. Лысьва, ул. Металлистов, 8	5904789012	Козлова Наталья Сергеевна	+7(902)111-22-35	kozlova@mail.ru	63

Рисунок 7 – Таблица partners_zaitova с данными (pgAdmin).

Как видно из рисунка, в таблице содержатся данные о семи партнерах из Перми и Пермского края: ООО "ПермьПаркет", ИП Соболева Е.В., ООО "УралПол", ЗАО "СтройКомплект", ИП Чайкин Д.А., ООО "ПармаСтрой" и ИП Козлова Н.С. Для каждого партнера указаны тип, наименование, юридический адрес, ИНН, ФИО директора, контактные данные и рейтинг.

Для хранения информации о продукции была создана таблица products_zaitova, структура которой показана на Рисунке 8.

id [PK] integer	article character varying (50)	name character varying (255)	description text	min_price numeric (15,2)	unit character varying (20)
1	MP-001	Ламинат Дуб благородный	Ламинат 32 класс, толщина 8 мм	850.00	м²
2	MP-002	Паркетная доска Ясень	Паркетная доска трехслойная, 14 мм	2100.00	м²
3	MP-003	Виниловая плитка	Кварц-винил, замковое соединение	1200.00	м²
4	MP-004	Плинтус напольный	Пластиковый, цвет белый	150.00	шт
5	MP-005	Подложка пробковая	Подложка 2 мм, рулон 10 м²	300.00	рулон

Рисунок 8 – Таблица products_zaitova с данными (pgAdmin).

В таблице продуктов содержатся пять наименований напольных покрытий: ламинат, паркетная доска, виниловая плитка, плинтус и подложка. Каждый продукт имеет уникальный артикул, описание, минимальную цену и единицу измерения.

Наиболее важной для функционала приложения является таблица истории продаж sales_history_zaitova. На Рисунке 9 представлены данные о продажах для всех партнеров.

	id [PK] integer	partner_id integer	product_id integer	quantity integer	sale_date date	total_amount numeric (15,2)	created_at timestamp without time zone
1	1	1	1	150	2026-01-10	127500.00	2026-03-15 13:20:09.138052
2	2	1	2	60	2026-01-15	126000.00	2026-03-15 13:20:09.138052
3	3	1	3	40	2026-01-25	48000.00	2026-03-15 13:20:09.138052
4	4	1	4	300	2026-02-01	45000.00	2026-03-15 13:20:09.138052
5	5	1	1	50	2026-02-10	42500.00	2026-03-15 13:20:09.138052
6	6	2	3	50	2026-01-12	60000.00	2026-03-15 13:20:09.138052
7	7	2	5	100	2026-01-20	30000.00	2026-03-15 13:20:09.138052
8	8	2	1	40	2026-02-05	34000.00	2026-03-15 13:20:09.138052
9	9	2	4	20	2026-02-15	3000.00	2026-03-15 13:20:09.138052
10	10	3	2	45	2026-01-08	94500.00	2026-03-15 13:20:09.138052
11	11	3	1	60	2026-01-18	51000.00	2026-03-15 13:20:09.138052
12	12	3	4	200	2026-01-28	30000.00	2026-03-15 13:20:09.138052
13	13	3	3	40	2026-02-07	48000.00	2026-03-15 13:20:09.138052
14	14	4	2	150	2026-01-05	315000.00	2026-03-15 13:20:09.138052
15	15	4	1	200	2026-01-22	170000.00	2026-03-15 13:20:09.138052
16	16	4	3	80	2026-02-03	96000.00	2026-03-15 13:20:09.138052
17	17	4	2	50	2026-02-12	105000.00	2026-03-15 13:20:09.138052
18	18	4	4	10	2026-02-18	1500.00	2026-03-15 13:20:09.138052
19	22	6	1	120	2026-01-17	102000.00	2026-03-15 13:20:09.138052
20	23	6	2	50	2026-01-27	105000.00	2026-03-15 13:20:09.138052

Рисунок 9 – Таблица sales_history_zaitova с данными (pgAdmin)

В таблице содержится более 20 записей о продажах, каждая из которых включает ссылку на партнера (partner_id), ссылку на продукт (product_id), количество, дату продажи и общую сумму. Эти данные используются для расчета индивидуальной скидки каждого партнера в соответствии с формулой из технического задания.

3. Документирование системы

В ходе разработки использовались комментарии для пояснения неочевидных фрагментов кода, например, в методе расчета скидки.

Пример комментирования кода (DiscountCalculator.cs):

```
public class DiscountCalculator : IDiscountCalculator
{
    public int CalculateDiscount(decimal totalSalesAmount)
    {
        if (totalSalesAmount < 10000)
            return 0;
        else if (totalSalesAmount < 50000)
            return 5;
        else if (totalSalesAmount < 300000)
            return 10;
        else
            return 15;
    }
}
```

4. Тестирование

Для проверки работоспособности ключевых компонентов были написаны модульные тесты (см. раздел 3.1.6). Тестирование проводилось после каждой итерации разработки.

5. Сопровождение системы

После завершения разработки модуль готов к внедрению. В случае возникновения ошибок предусмотрена возможность доработки и обновления.

Разработка форм для ввода, просмотра и редактирования данных

Были разработаны следующие формы:

1. Главное окно **(**MainWindow.xaml**)** – содержит список партнеров (левая колонка) и историю заказов выбранного партнера (правая колонка).

Интерфейс выполнен в соответствии с эскизом (Рис. 2 из задания). Внешний вид окна представлен на Рисунке 10 и 11.

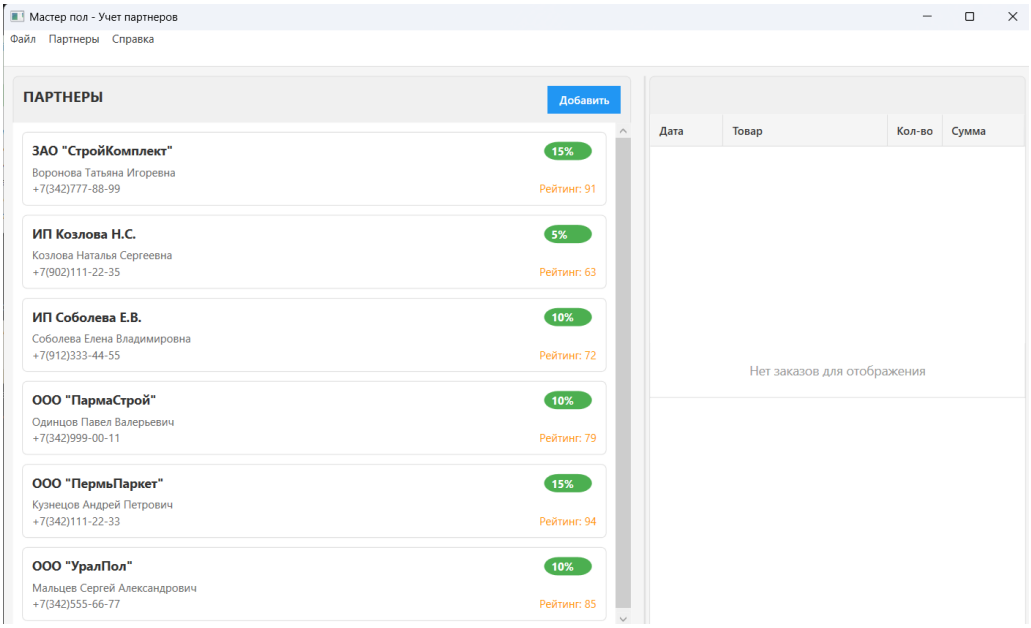

Рисунок 10 – Главное окно приложения.

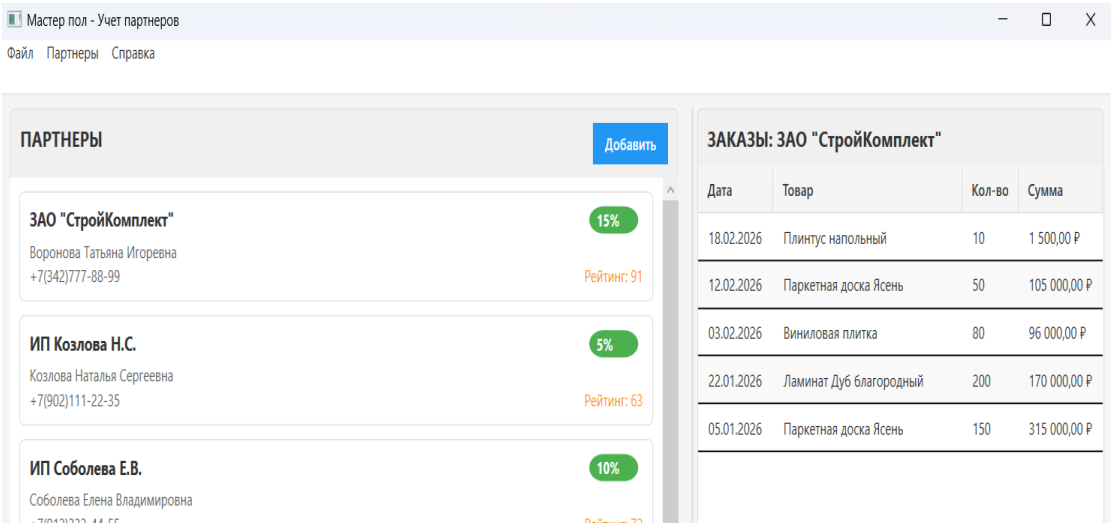

Рисунок 11 – Главное окно с выбранным партнером и историей заказов.

XAML-код главного окна (фрагмент):

```
<!-- Левая колонка: список партнеров -->
<ScrollViewer Grid.Column="0" VerticalScrollBarVisibility="Auto">
    <ItemsControl ItemsSource="{Binding Partners}">
        <ItemsControl.ItemsPanel>
            <ItemsPanelTemplate>
                <StackPanel Orientation="Vertical"/>
            </ItemsPanelTemplate>
        </ItemsControl.ItemsPanel>
        <ItemsControl.ItemTemplate>
            <DataTemplate>
```

```
                <Border BorderBrush="#E0E0E0"
                        BorderThickness="1"
                        CornerRadius="5"
                        Padding="15"
                        Margin="0,0,0,10"
                        Background="{Binding        IsSelected,        Convert-
er={StaticResource SelectedToBackgroundConverter}}"
                        Tag="{Binding}"
                        MouseLeftButtonUp="Border_MouseLeftButtonUp"
                        Cursor="Hand">
                    <Grid>
                        <Grid.RowDefinitions>
                            <RowDefinition Height="Auto"/>
                            <RowDefinition Height="Auto"/>
                            <RowDefinition Height="Auto"/>
                        </Grid.RowDefinitions>
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="Auto"/>
                        </Grid.ColumnDefinitions>

                        <!-- Тип | Название -->
                        <TextBlock Grid.Row="0" Grid.Column="0"
                                   FontSize="14" FontWeight="Bold"
                                   Foreground="#333333"
                                   Text="{Binding CompanyName}"/>

                        <!-- Скидка -->
                        <Border Grid.Row="0" Grid.Column="1"
                                Background="#4CAF50" CornerRadius="15"
                                Padding="8,2" Margin="5,0">
                            <TextBlock    Text="{Binding    DiscountPercentage,
StringFormat='{}{0}%'}"
                                       Foreground="White" FontWeight="Bold"/>
                        </Border>

                        <!-- Директор -->
                        <TextBlock         Grid.Row="1"         Grid.Column="0"
Grid.ColumnSpan="2"
                                   Text="{Binding DirectorFullname}"
                                   Foreground="#666666" FontSize="12"
                                   Margin="0,5,0,2"/>

                        <!-- Телефон -->
                        <StackPanel   Grid.Row="2"   Grid.Column="0"   Orienta-
tion="Horizontal">
                            <TextBlock Text="{Binding Phone}"
                                       Foreground="#666666" FontSize="12"/>
                        </StackPanel>

                        <!-- Рейтинг -->
                        <TextBlock Grid.Row="2" Grid.Column="1"
                                   Text="{Binding       Rating,       StringFor-
mat='Рейтинг: {0}'}"
                                   Foreground="#FF9800" FontSize="12"/>
                    </Grid>
                </Border>
            </DataTemplate>
        </ItemsControl.ItemTemplate>
    </ItemsControl>
</ScrollViewer>
```

2. Окно добавления партнера **(**PartnerEditWindow.xaml**)** – содержит все необходимые поля: наименование, тип партнера (выпадающий список), рейтинг, адрес, ФИО директора, телефон, email. Внешний вид окна представлен на Рисунке 12.

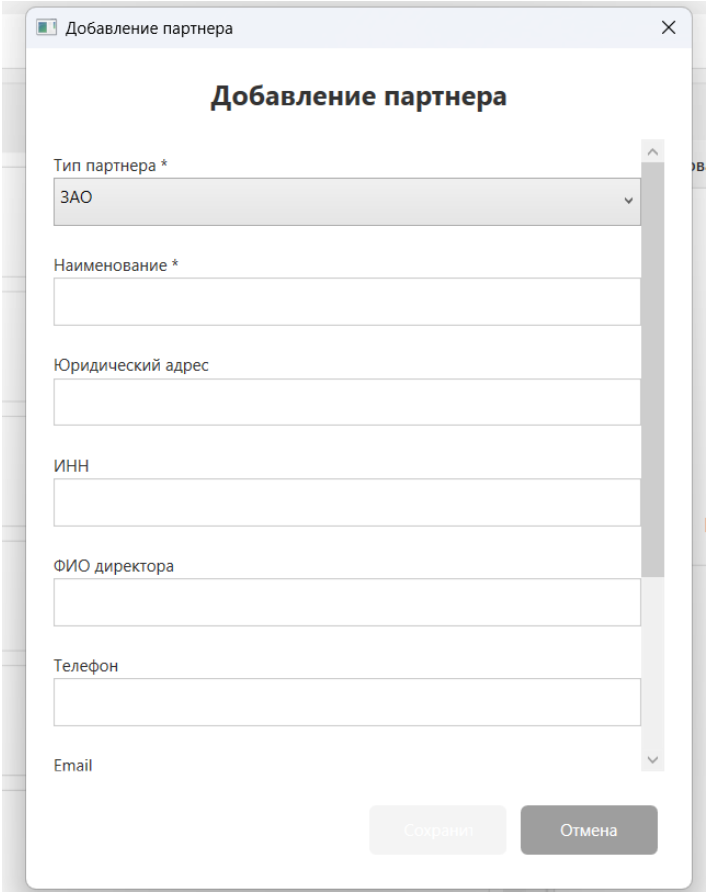

Рисунок 12. – Окно добавления партнера.

На форме реализованы:

- Выпадающий список типов партнеров (загружается из БД).
- Валидация полей (рейтинг от 0 до 100, обязательные поля).
- Подсказка для пользователя внизу формы.

При открытии формы в режиме редактирования все поля автоматически заполняются данными выбранного партнера (см. Рисунок 13).

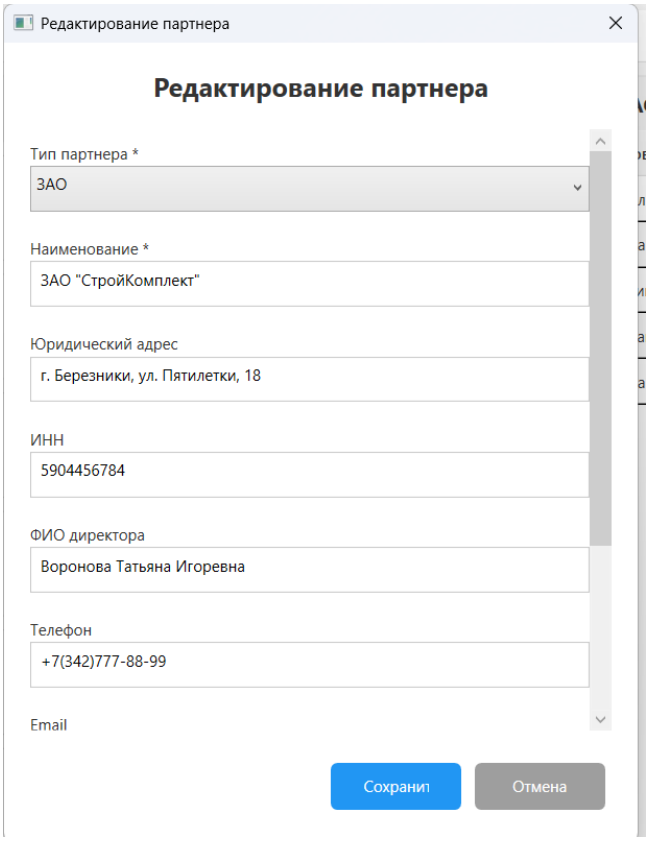

Рисунок 13 – Режим редактирования партнера.

XAML-код окна редактирования (фрагмент):

```
<!-- Тип партнера -->
<TextBlock Text="Тип партнера *" Margin="5,10,5,2" Foreground="#333333"/>
<ComboBox ItemsSource="{Binding PartnerTypes}"
          SelectedItem="{Binding SelectedType}"
          DisplayMemberPath="Name"
          Height="35" Margin="0,0,0,10" Padding="5"/>

<!-- Наименование -->
<TextBlock Text="Наименование *" Margin="5,10,5,2" Foreground="#333333"/>
<TextBox  Text="{Binding  CurrentPartner.CompanyName,  UpdateSourceTrig-
ger=PropertyChanged}"
          Height="35" Margin="0,0,0,10" Padding="5"/>

<!-- Юридический адрес -->
<TextBlock    Text="Юридический    адрес"    Margin="5,10,5,2"    Fore-
ground="#333333"/>
<TextBox Text="{Binding CurrentPartner.LegalAddress}"
         Height="35" Margin="0,0,0,10" Padding="5"/>

<!-- ИНН -->
<TextBlock Text="ИНН" Margin="5,10,5,2" Foreground="#333333"/>
<TextBox Text="{Binding CurrentPartner.Inn}"
         Height="35" Margin="0,0,0,10" Padding="5" MaxLength="12"/>

<!-- ФИО директора -->
<TextBlock Text="ФИО директора" Margin="5,10,5,2" Foreground="#333333"/>
<TextBox Text="{Binding CurrentPartner.DirectorFullname}"
         Height="35" Margin="0,0,0,10" Padding="5"/>

<!-- Телефон -->
<TextBlock Text="Телефон" Margin="5,10,5,2" Foreground="#333333"/>
```

```
        <TextBox Text="{Binding CurrentPartner.Phone}"
                 Height="35" Margin="0,0,0,10" Padding="5"/>

        <!-- Email -->
        <TextBlock Text="Email" Margin="5,10,5,2" Foreground="#333333"/>
        <TextBox Text="{Binding CurrentPartner.Email}"
                 Height="35" Margin="0,0,0,10" Padding="5"/>

        <!-- Рейтинг -->
        <TextBlock    Text="Рейтинг    (0-100)    *"    Margin="5,10,5,2"    Fore-
ground="#333333"/>
        <TextBox Text="{Binding CurrentPartner.Rating}"
                 Height="35" Margin="0,0,0,10" Padding="5"/>
```

3. Окно истории продаж (SalesHistoryWindow.xaml**)** – использовалось на ранних этапах разработки, позже история была интегрирована в главное окно. Оставлено для совместимости и возможности просмотра расширенной информации.

ЗАКАЗЫ: ЗАО "СтройКомплект"

Дата	Товар	Кол-во	Сумма
18.02.2026	Плинтус напольный	10	1 500,00 ₽
12.02.2026	Паркетная доска Ясень	50	105 000,00 ₽
03.02.2026	Виниловая плитка	80	96 000,00 ₽
22.01.2026	Ламинат Дуб благородный	200	170 000,00 ₽
05.01.2026	Паркетная доска Ясень	150	315 000,00 ₽

Рисунок 14 – Окно истории продаж (отдельное окно).

Этапы разработки интерфейса:

1. Проектирование

На основе эскиза (Рис. 2 из задания) был разработан макет интерфейса в виде наброска на бумаге, затем перенесен в XAML.

2. Прототипирование

Создан прототип в XAML с базовой версткой: расположены основные элементы (ListBox для партнеров, DataGrid для истории, кнопки). На этом этапе проверялась функциональность привязки данных (Binding).

3. Стилизация

После проверки работоспособности проведена стилизация интерфейса: заданы цвета, шрифты, отступы, рамки. Стили вынесены в ресурсы приложения (App.xaml) для обеспечения единообразного внешнего вида всех окон.

Пример стилей из App.xaml:

```
<Style x:Key="DataGridCellStyle" TargetType="TextBlock">
    <Setter Property="VerticalAlignment" Value="Center"/>
    <Setter Property="Margin" Value="10,0"/>
    <Setter Property="FontSize" Value="12"/>
    <Setter Property="Foreground" Value="#333333"/>
</Style>

<Style x:Key="DataGridHeaderStyle" TargetType="DataGridColumnHeader">
    <Setter Property="Background" Value="#F5F5F5"/>
    <Setter Property="Foreground" Value="#333333"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="Height" Value="35"/>
    <Setter Property="BorderBrush" Value="#E0E0E0"/>
    <Setter Property="BorderThickness" Value="0,0,1,1"/>
</Style>
```

3.2.3. Использование методов для получения кода с заданной функциональностью и степенью качества

Для обеспечения высокого качества кода и требуемой функциональности применялись следующие методы:

1. Принципы SOLID:

- Single Responsibility Principle – каждый класс отвечает за одну функциональность (например, PartnerRepository – только работа с БД, DiscountCalculator – только расчет скидки).
- Open/Closed Principle – классы открыты для расширения, но закрыты для модификации (использование интерфейсов).
- Liskov Substitution Principle – наследующие классы могут заменять базовые (например, PartnerRepository реализует IPartnerRepository).
- Interface Segregation Principle – интерфейсы разделены по функциональности (IPartnerRepository, IProductRepository).
- Dependency Inversion Principle – зависимости строятся на абстракциях (внедрение интерфейсов в конструкторы).

- Паттерн Repository – абстрагирует слой доступа к данным, позволяет легко менять источник данных (например, заменить PostgreSQL на другой СУБД).

Пример использования паттерна Repository:

```
// Интерфейс репозитория
public interface IPartnerRepository
{
    Task<List<Partner>> GetAllAsync();
    Task<Partner?> GetByIdAsync(int id);
    Task<Partner> AddAsync(Partner partner);
    Task UpdateAsync(Partner partner);
    Task DeleteAsync(int id);
}

// Реализация репозитория
public class PartnerRepository : IPartnerRepository
{
    private readonly AppDbContext _context;

    public PartnerRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<Partner>> GetAllAsync()
    {
        return await _context.Partners
            .Include(p => p.PartnerType)
            .OrderBy(p => p.CompanyName)
            .ToListAsync();
    }

    // ... остальные методы
}
```

- Паттерн Dependency Injection – зависимости передаются через конструктор, что упрощает тестирование и делает код гибким.

Внедрение зависимостей через конструктор:

```
public class PartnerService : IPartnerService
{
    private readonly IPartnerRepository _partnerRepository;
    private readonly IProductRepository _productRepository;
    private readonly IPartnerTypeRepository _partnerTypeRepository;
    private readonly IDiscountCalculator _discountCalculator;

    public PartnerService(
        IPartnerRepository partnerRepository,
        IProductRepository productRepository,
        IPartnerTypeRepository partnerTypeRepository,
        IDiscountCalculator discountCalculator)
```

```
        {
            _partnerRepository = partnerRepository;
            _productRepository = productRepository;
            _partnerTypeRepository = partnerTypeRepository;
            _discountCalculator = discountCalculator;
        }

        // ... методы сервиса
    }
```

- Именование идентификаторов – все переменные, методы и классы названы осмысленно, в соответствии с соглашением CamelCase (для C#):
- Классы: PartnerService, DiscountCalculator – существительные с заглавной буквы.
- Методы: GetAllPartnersAsync, CalculateDiscount – глаголы с заглавной буквы.
- Параметры и локальные переменные: partnerId, totalSalesAmount – с маленькой буквы, camelCase.

3.2.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Виды инспекции:

1. Самопроверка (self-review) – авторская проверка кода перед коммитом: анализ стиля, удаление лишних комментариев, проверка соответствия именования.
2. Формальная инспекция – проверка кода руководителем практики (совместный просмотр, выявление недочетов).

Актуальные стандарты кодирования:

- Соглашение об именовании в C# (Microsoft Naming Guidelines):
  - PascalCase для классов, методов, свойств.
  - camelCase для параметров, локальных переменных, полей (с префиксом _).
- Форматирование кода – отступы (4 пробела), фигурные скобки на новой строке.

- Комментирование – комментарии только в сложных местах (метод расчета скидки), XML-комментарии для публичных API.
- Обработка исключений – все места, где возможны ошибки (работа с БД), обернуты в try-catch с информированием пользователя.

Инструменты инспекции:

- Visual Studio 2022 – встроенный анализатор кода (Code Analysis) подсвечивает потенциальные проблемы, несоответствия стиля.
- Roslyn-анализаторы – правила проверки кода, включенные в проект.
- Git – история изменений позволяет отслеживать, кто и когда вносил правки.

3.2.5. Выполнение отладки программного модуля с использованием специализированных программных средств

Отладка проводилась с использованием встроенного отладчика Visual Studio 2022. Процесс отладки включал следующие этапы:

1. **Установка точек останова (breakpoints)** – для остановки выполнения программы в критических местах: при загрузке данных, при сохранении, при расчете скидки.
2. **Пошаговое выполнение (Step Into / Step Over)** – для отслеживания последовательности выполнения кода, проверки логики.
3. **Окно просмотра значений (Watch)** – для наблюдения за значениями переменных в реальном времени.
4. **Обработка исключений** – настроена остановка при возникновении исключений, что позволило выявить и исправить ошибку с загрузкой истории продаж (отсутствие данных в БД).
5. **Вывод отладочной информации** – использование System.Diagnostics.Debug.WriteLine() для вывода в окно Output.

Вывод отладочной информации:

```
System.Diagnostics.Debug.WriteLine($"Загружено записей истории: {history.Count}");
foreach (var item in history)
{
    System.Diagnostics.Debug.WriteLine($"  - {item.ProductName}: {item.Quantity} шт, {item.TotalAmount} руб");
}
```

В результате отладки были выявлены и устранены следующие проблемы:

- Ошибка привязки данных в DataGrid (неправильные имена свойств).
- Проблема с загрузкой истории при первом выборе партнера.
- Ошибка при расчете скидки для новых партнеров (деление на ноль).

3.2.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

Для проверки корректности работы ключевых компонентов были разработаны модульные тесты с использованием фреймворка **MSTest**. Тесты покрывают наиболее важные части системы: расчет скидки, работу сервиса партнеров, репозитория.

Unit-тесты для DiscountCalculator:

Тесты для DiscountCalculator (DiscountCalculatorTests.cs):

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using ZaitovaLibrary.Services;

namespace Zaitova.Tests
{
    [TestClass]
    public class DiscountCalculatorTests
    {
        private DiscountCalculator _calculator;

        [TestInitialize]
        public void Setup()
        {
            _calculator = new DiscountCalculator();
        }

        [TestMethod]
        public void CalculateDiscount_TotalLessThan10000_Returns0()
        {
            // Arrange
            decimal totalSales = 5000;
```

```
                // Act
                int discount = _calculator.CalculateDiscount(totalSales);

                // Assert
                Assert.AreEqual(0, discount);
            }

            [TestMethod]
            public void CalculateDiscount_TotalEqualTo10000_Returns5()
            {
                // Arrange
                decimal totalSales = 10000;

                // Act
                int discount = _calculator.CalculateDiscount(totalSales);

                // Assert
                Assert.AreEqual(5, discount);
            }

            [TestMethod]
            public                      void                      CalculateDis-
count_TotalBetween10000And50000_Returns5()
            {
                // Arrange
                decimal totalSales = 25000;

                // Act
                int discount = _calculator.CalculateDiscount(totalSales);

                // Assert
                Assert.AreEqual(5, discount);
            }

            [TestMethod]
            public void CalculateDiscount_TotalEqualTo50000_Returns10()
            {
                // Arrange
                decimal totalSales = 50000;

                // Act
                int discount = _calculator.CalculateDiscount(totalSales);

                // Assert
                Assert.AreEqual(10, discount);
            }

            [TestMethod]
            public                      void                      CalculateDis-
count_TotalBetween50000And300000_Returns10()
            {
                // Arrange
                decimal totalSales = 150000;

                // Act
                int discount = _calculator.CalculateDiscount(totalSales);

                // Assert
                Assert.AreEqual(10, discount);
            }

            [TestMethod]
            public void CalculateDiscount_TotalEqualTo300000_Returns15()
```

```
        {
            // Arrange
            decimal totalSales = 300000;

            // Act
            int discount = _calculator.CalculateDiscount(totalSales);

            // Assert
            Assert.AreEqual(15, discount);
        }

        [TestMethod]
        public void CalculateDiscount_TotalMoreThan300000_Returns15()
        {
            // Arrange
            decimal totalSales = 500000;

            // Act
            int discount = _calculator.CalculateDiscount(totalSales);

            // Assert
            Assert.AreEqual(15, discount);
        }

        [TestMethod]
        public void CalculateDiscount_TotalZero_Returns0()
        {
            // Arrange
            decimal totalSales = 0;

            // Act
            int discount = _calculator.CalculateDiscount(totalSales);

            // Assert
            Assert.AreEqual(0, discount);
        }

        [TestMethod]
        public void CalculateDiscount_TotalNegative_Returns0()
        {
            // Arrange
            decimal totalSales = -1000;

            // Act
            int discount = _calculator.CalculateDiscount(totalSales);

            // Assert
            Assert.AreEqual(0, discount);
        }
    }
}
```

Unit-тесты для PartnerRepository (с in-memory БД):

Для тестирования репозитория использовалась in-memory база данных Entity Framework Core, что позволяет не зависеть от реальной PostgreSQL.

Тесты для PartnerRepository (PartnerRepositoryTests.cs):

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Microsoft.EntityFrameworkCore;
using ZaitovaLibrary.Data;
using ZaitovaLibrary.Models;
using System.Threading.Tasks;
using System.Linq;

namespace Zaitova.Tests
{
    [TestClass]
    public class PartnerRepositoryTests
    {
        private AppDbContext _context;
        private PartnerRepository _repository;

        [TestInitialize]
        public void Setup()
        {
            // Создаем уникальное имя базы для каждого теста
            var databaseName = $"ZaitovaTestDb_{System.Guid.NewGuid()}";

            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(databaseName: databaseName)
                .Options;

            _context = new AppDbContext(options);
            _repository = new PartnerRepository(_context);

            // Добавляем тестовые данные
            SeedData();
        }

        private void SeedData()
        {
            // Добавляем тип партнера
            if (!_context.PartnerTypes.Any())
            {
                _context.PartnerTypes.Add(new PartnerType
                {
                    Id = 1,
                    Name = "ООО"
                });
                _context.SaveChanges();
            }
        }

        [TestCleanup]
        public void Cleanup()
        {
            _context.Database.EnsureDeleted();
            _context.Dispose();
        }

        [TestMethod]
        public async Task AddAsync_ValidPartner_ReturnsPartnerWithId()
        {
            // Arrange
            var partner = new Partner
            {
                TypeId = 1,
                CompanyName = "ООО Тест",
                DirectorFullname = "Иванов Иван Иванович",
                Phone = "+7(123)456-78-90",
```

```
            Email = "test@test.ru",
            Rating = 50,
            CreatedAt = System.DateTime.Now,
            UpdatedAt = System.DateTime.Now
        };

        // Act
        var result = await _repository.AddAsync(partner);

        // Assert
        Assert.IsNotNull(result);
        Assert.IsTrue(result.Id > 0);
        Assert.AreEqual("ООО Тест", result.CompanyName);
    }

    [TestMethod]
    public async Task GetByIdAsync_ExistingId_ReturnsPartner()
    {
        // Arrange
        var partner = new Partner
        {
            TypeId = 1,
            CompanyName = "ООО Тест",
            DirectorFullname = "Иванов Иван Иванович",
            Rating = 50,
            CreatedAt = System.DateTime.Now,
            UpdatedAt = System.DateTime.Now
        };

        var added = await _repository.AddAsync(partner);

        // Act
        var result = await _repository.GetByIdAsync(added.Id);

        // Assert
        Assert.IsNotNull(result);
        Assert.AreEqual(added.Id, result.Id);
        Assert.AreEqual("ООО Тест", result.CompanyName);
    }

    [TestMethod]
    public async Task UpdateAsync_ExistingPartner_UpdatesProperties()
    {
        // Arrange
        var partner = new Partner
        {
            TypeId = 1,
            CompanyName = "Старое название",
            Rating = 50,
            CreatedAt = System.DateTime.Now,
            UpdatedAt = System.DateTime.Now
        };

        var added = await _repository.AddAsync(partner);

        // Act
        added.CompanyName = "Новое название";
        added.Rating = 75;
        await _repository.UpdateAsync(added);

        // Assert
        var updated = await _repository.GetByIdAsync(added.Id);
        Assert.IsNotNull(updated);
```

```
            Assert.AreEqual("Новое название", updated.CompanyName);
            Assert.AreEqual(75, updated.Rating);
        }

        [TestMethod]
        public async Task DeleteAsync_ExistingId_RemovesPartner()
        {
            // Arrange
            var partner = new Partner
            {
                TypeId = 1,
                CompanyName = "ООО Для удаления",
                Rating = 50,
                CreatedAt = System.DateTime.Now,
                UpdatedAt = System.DateTime.Now
            };

            var added = await _repository.AddAsync(partner);

            // Act
            await _repository.DeleteAsync(added.Id);
            var result = await _repository.GetByIdAsync(added.Id);

            // Assert
            Assert.IsNull(result);
        }
    }
}
```

Результаты выполнения тестов представлены на Рисунке 15 – все тесты пройдены успешно.

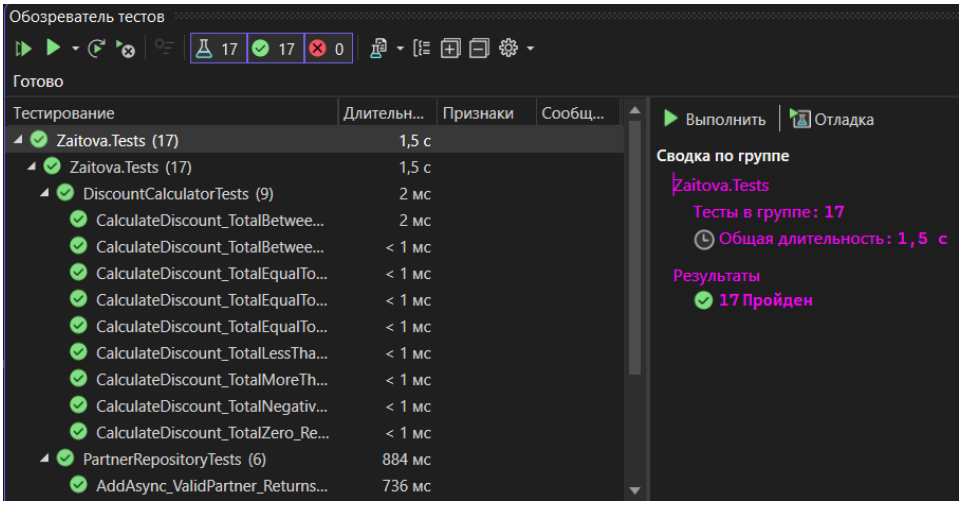

Рисунок 15 – Результаты выполнения модульных тестов.

3.2.7. Интегрирование программных модулей

Интеграция модулей выполнялась поэтапно, в соответствии с итерационной моделью разработки:

1. **Интеграция слоя доступа к данным** – подключение AppDbContext к PostgreSQL, настройка строки подключения в appsettings.json.

Строка подключения к БД (appsettings.json):

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Host=localhost;Port=5432;Database=Zaitova_BD;Username=postgres;Password=postgr
es"
  }
}
```

2. **Интеграция репозиториев** – созданы репозитории (PartnerRepository, ProductRepository), которые используют AppDbContext. Проверено выполнение базовых операций (CRUD).

3. **Интеграция сервисов** – создан PartnerService, который использует репозитории и DiscountCalculator. Все зависимости передаются через конструктор (Dependency Injection), настройка DI контейнера выполнена в App.xaml.cs.

Настройка DI контейнера (App.xaml.cs):

```
private void ConfigureServices(IServiceCollection services)
{
    services.AddDbContext<AppDbContext>(options =>
        op-
tions.UseNpgsql(Configuration.GetConnectionString("DefaultConnection")));

    services.AddScoped<IPartnerRepository, PartnerRepository>();
    services.AddScoped<IProductRepository, ProductRepository>();
    services.AddScoped<IPartnerTypeRepository, PartnerTypeRepository>();
    services.AddScoped<IDiscountCalculator, DiscountCalculator>();
    services.AddScoped<IPartnerService, PartnerService>();
    services.AddTransient<MainViewModel>();
    services.AddTransient<PartnerEditViewModel>();
}
```

4. **Интеграция слоя представления** – ViewModel'и (MainViewModel, PartnerEditViewModel) получают сервисы через DI и связываются с XAML через привязку данных (Data Binding).

5. **Интеграция истории продаж** – при выборе партнера в MainViewModel автоматически загружается история продаж через PartnerService и отображается в DataGrid справа.

Загрузка истории при выборе партнера (MainViewModel.cs):

```
public PartnerDto? SelectedPartner
{
    get => _selectedPartner;
    set
    {
        if (SetProperty(ref _selectedPartner, value))
        {
            if (value != null)
            {
                LoadSalesHistoryAsync(value.Id);
            }
            else
            {
                SelectedPartnerOrders.Clear();
            }
        }
    }
}
```

Интеграция всех модулей прошла успешно, приложение работает стабильно.

3.2.8. Верификация и аттестация программного обеспечения

Верификация (проверка) и аттестация (оценка соответствия) программного обеспечения проводились на заключительном этапе разработки.

Процесс верификации:

1. **Проверка соответствия требованиям ТЗ** – сверка реализованного функционала с заданием:
   - Просмотр списка партнеров – реализован.
   - Добавление/редактирование – реализовано.
   - Просмотр истории продаж – реализован (справа от списка партнеров).
2. **Проверка пользовательского интерфейса** – соответствие эскизу (Рис. 2 из задания):

- Список партнеров слева.
- История заказов справа.
- Кнопки управления в меню.

3. **Проверка обработки ошибок** – протестированы следующие ситуации:

- Попытка удалить партнера без подтверждения – появляется предупреждение (см. Рисунок 16).
- Ввод некорректных данных (рейтинг > 100) – при сохранении появляется сообщение об ошибке.
- Отсутствие подключения к БД – сообщение об ошибке при запуске.

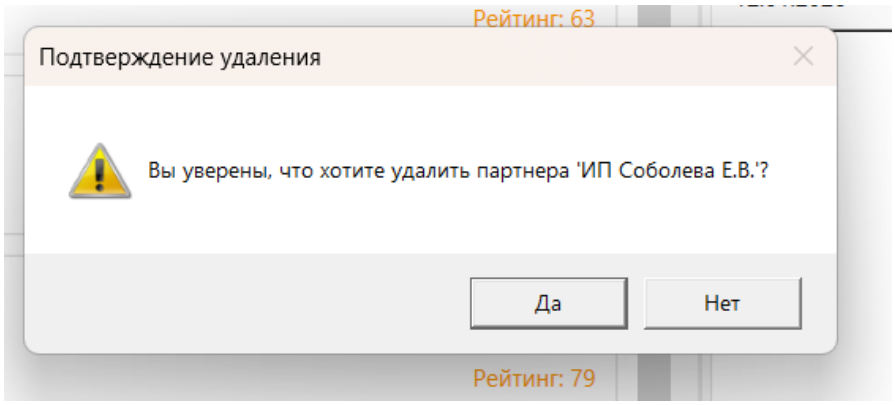

Рисунок 16 – Предупреждение перед удалением партнера.

4. **Проверка расчета скидки** – для партнеров с разными суммами продаж рассчитаны скидки (см. Таблицу 3).

Таблица 3 – Проверка расчета скидки.

Партнер	Сумма продаж	Ожидаемая скидка	Фактическая скидка
ООО "ПермьПаркет"	389 000 ₽	15%	15%
ИП Соболева Е.В.	127 000 ₽	10%	10%
ИП Чайкин Д.А.	39 000 ₽	5%	5%

Как видно из таблицы, для всех тестовых партнеров фактическая скидка полностью соответствует ожидаемой, рассчитанной по формуле из технического задания. Это подтверждает корректность работы алгоритма DiscountCalculator. В ходе тестирования были проверены все диапазоны сумм продаж: менее 10 000 ₽ (скидка 0%), от 10 000 до 50 000 ₽ (скидка 5%), от 50 000 до 300 000 ₽ (скидка 10%) и более 300 000 ₽ (скидка 15%).

Дополнительно было проведено функциональное тестирование приложения, включающее проверку навигации по меню, работы со списком партнеров, операций добавления, редактирования и удаления, а также корректности отображения истории продаж. Все проверки завершились успешно.

Аттестация:

По результатам верификации и тестирования программный модуль признан соответствующим требованиям задания. Все функции работают корректно, исключительные ситуации обрабатываются, интерфейс соответствует эскизу. Модуль готов к внедрению в эксплуатацию.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики на математическом факультете ПГГПУ с 09 по 17 марта 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Разработана подсистема для работы с партнерами компании «Мастер пол», реализующая весь требуемый функционал:
   - просмотр списка партнеров;
   - добавление и редактирование данных о партнере;
   - просмотр истории реализации продукции.
2. Создана база данных в PostgreSQL, приведенная к 3 нормальной форме, с соблюдением ссылочной целостности. Все объекты БД содержат фамилию студента (_zaitova).
3. Получена ER-диаграмма базы данных и диаграмма классов разработанного приложения.
4. Написаны модульные тесты для ключевых компонентов (DiscountCalculator, PartnerRepository), подтверждающие корректность работы алгоритмов.
5. Проведена отладка и верификация программного модуля, подтверждено соответствие требованиям технического задания.
6. Изучена и применена система контроля версий Git, код размещен на GitHub.
7. Приобретены навыки работы в профессиональном коллективе, ведения технической документации.

Особое внимание в ходе разработки уделялось качеству кода и его соответствию стандартам. Применялись принципы SOLID, паттерны Repository и Dependency Injection, соглашения об именовании CamelCase и PascalCase. Все публичные методы снабжены XML-комментариями, сложные участки кода пояснены inline-комментариями.

Разработанное приложение имеет интуитивно понятный интерфейс, соответствующий эскизу из технического задания. Реализована подсветка выбранного партнера, автоматическая загрузка истории продаж, информативные сообщения об ошибках и предупреждениях. Интерфейс выполнен в едином стиле, все окна приложения имеют соответствующие заголовки.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования; работы с системами контроля версий; проектирования баз данных и работы с СУБД PostgreSQL.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств».

2. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.: Издательский дом «Академия», 2016 – 304с.

3. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. – М.: ООО "Научно-издательский центр ИНФРА-М", 2013. – 400 с.

4. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. – М.: ИД ФОРУМ: ИНФРА-М, 2012. – 240 с.

5. Немцова Т.И. Программирование на языке высокого уровня. Программирование на языке C#: Учебное пособие / Т.И. Немцова, С.Ю. Голова, И.В. Абрамов; Под ред. Л.Г. Гагариной. – М.: ИД ФОРУМ: НИЦ ИНФРА-М, 2015. – 512 с.

6. Методические материалы по PostgreSQL [Электронный ресурс]. – URL: https://metanit.com/sql/postgresql/

7. Документация по Entity Framework Core [Электронный ресурс]. – URL: https://learn.microsoft.com/ru-ru/ef/core/

8. Руководство по языку C# [Электронный ресурс]. – URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/

9. Паттерны проектирования [Электронный ресурс]. – URL: https://metanit.com/sharp/patterns/

10. Официальный сайт ПГГПУ [Электронный ресурс]. – URL: https://pspu.ru/

11. Руководство по языку C# [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 15.03.2026).

12. Документация по WPF [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 15.03.2026).

13. Паттерны проектирования [Электронный ресурс]. – Режим доступа: https://metanit.com/sharp/patterns/ (дата обращения: 15.03.2026).

14. Троелсен Э., Джепикс Ф. Язык программирования C# 7 и платформы .NET и .NET Core. – М.: Вильямс, 2018. – 1328 с.

15. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. – СПб.: Питер, 2017. – 896 с.